Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3637871

PDF Download
3637871.pdf
23 January 2026
Total Citations: 35
Total Downloads: 1869

Published: 22 January 2024
Online AM: 18 December 2023
Accepted: 28 November 2023
Revised: 22 September 2023
Received: 08 January 2023

Citation in BibTeX format

RESEARCH-ARTICLE

SMLP4Rec: An Efficient All-MLP Architecture for Sequential Recommendations

JINGTONG GAO, City University of Hong Kong, Hong Kong, Hong Kong

XIANGYU ZHAO, City University of Hong Kong, Hong Kong, Hong Kong

MUYANG LI, The University of Sydney, Sydney, NSW, Australia

MINGHAO ZHAO

RUNZE WU

RUOCHENG GUO, ByteDance Ltd., Beijing, China

View all

Open Access Support provided by:

Baidu, Inc.

The University of Sydney

City University of Hong Kong

ByteDance Ltd.

SMLP4Rec: An Efficient All-MLP Architecture for Sequential Recommendations

JINGTONG GAO and XIANGYU ZHAO, City University of Hong Kong
MUYANG LI, University of Sydney
MINGHAO ZHAO and RUNZE WU, NetEase FUXI AI Lab
RUOCHENG GUO, Bytedance AI Lab
YIDING LIU and DAWEI YIN, Baidu Inc

Self-attention models have achieved the state-of-the-art performance in sequential recommender systems by capturing the sequential dependencies among user–item interactions. However, they rely on adding positional embeddings to the item sequence to retain the sequential information, which may break the semantics of item embeddings due to the heterogeneity between these two types of embeddings. In addition, most existing works assume that such dependencies exist solely in the item embeddings, but neglect their existence among the item features. In our previous study, we proposed a novel sequential recommendation model, i.e., MLP4Rec, based on the recent advances of MLP-Mixer architectures, which is naturally sensitive to the order of items in a sequence because matrix elements related to different positions of a sequence will be given different weights in training. We developed a tri-directional fusion scheme to coherently capture sequential, cross-channel, and cross-feature correlations with linear computational complexity as well as much fewer model parameters than existing self-attention methods. However, the cascading mixer structure, the large number of normalization layers between different mixer layers, and the noise generated by these operations limit the efficiency of information extraction and the effectiveness of MLP4Rec. In this extended version, we propose a novel framework – SMLP4Rec for sequential recommendation to address the aforementioned issues. The new framework changes the flawed cascading structure to a parallel mode, and integrates normalization layers to minimize their impact on the model's efficiency while maximizing their effectiveness. As a result, the training speed and prediction accuracy of SMLP4Rec are vastly improved in comparison to MLP4Rec. Extensive experimental results demonstrate that the proposed method is significantly superior to the state-of-the-art approaches. The implementation code is available online to ease reproducibility.

This research was partially supported by Research Impact Fund (No. R1015-23), APRC - CityU New Research Initiatives (No. 9610565, Start-up Grant for New Faculty of City University of Hong Kong), CityU - HKIDS Early Career Research Grant (No. 9360163), Hong Kong ITC Innovation and Technology Fund Midstream Research Programme for Universities Project (No. ITS/034/22MS), Hong Kong Environmental and Conservation Fund (No. 88/2022), and SIRG - CityU Strategic Interdisciplinary Research Grant (No. 7020046, No. 7020074), Huawei (Huawei Innovation Research Program), Ant Group (CCF-Ant Research Fund, Ant Group Research Fund), Tencent (CCF-Tencent Open Fund, Tencent Rhino-Bird Focused Research Program), CCF-BaiChuan-Ebtech Foundation Model Fund, Kuaishou, and Key Research and Development Program of Zhejiang Province (No. 2022C01011).
Authors' addresses: J. Gao and X. Zhao (Corresponding author), City University of Hong Kong, Hong Kong, Hong Kong; e-mails: jt.g@my.cityu.edu.hk, xianzhao@cityu.edu.hk; M. Li, University of Sydney, Sydney, Australia; e-mail: muli0371@uni.sydney.edu.au; M. Zhao and R. Wu, NetEase FUXI AI Lab, Hangzhou, China; e-mails: {zhaominghao, wurunze1}@corp.netease.com; R. Guo, Bytedance AI Lab London, United Kingdom; e-mail: rguo.asu@gmail.com; Y. Liu and D. Yin, Baidu Inc, Beijing, China; e-mails: liuyiding.tanh@gmail.com, yindawei@acm.org.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
1046-8188/2024/01-ART86 $15.00
https://doi.org/10.1145/3637871

CCS Concepts: • **Information systems → Retrieval models and ranking**; **Data mining;**

Additional Key Words and Phrases: Recommender system, sequential recommendation, multi-layer perceptron

ACM Reference format:
Jingtong Gao, Xiangyu Zhao, Muyang Li, Minghao Zhao, Runze Wu, Ruocheng Guo, Yiding Liu, and Dawei Yin. 2024. SMLP4Rec: An Efficient All-MLP Architecture for Sequential Recommendations. *ACM Trans. Inf. Syst.* 42, 3, Article 86 (January 2024), 23 pages.
https://doi.org/10.1145/3637871

1 INTRODUCTION

Accurately modeling the chronological behavior of users is a critical area of research in recommender systems. The primary challenge is to capture the sequential pattern of user interests across multiple items, which is typically dynamic. To address this issue, **Sequential Recommender Systems** (**SRS**) [20, 40, 42] were proposed and have garnered considerable interest from both academia and industry. While many endeavors have been put into this field, the newly emerged self-attention mechanism [33] has gained a dominant position in recent years. Specifically, the self-attention mechanism tries to aggregate sequential information by applying a position and similarity-based weighted sum operation on the items of an interaction sequence. By doing so, it allows the model to better understand the association of items and their positions in the sequence. Recent works show that the self-attention based models can significantly outperform other models, and achieve the **state-of-the-art** (**SOTA**) performance in SRS [6, 28, 41].

Despite the success of the self-attention mechanism in sequential recommendations, some limitations can potentially restrict its further development and practical applications. First, since commonly used item representations are unable to provide positional information of the items in the weighted sum operation, self-attention and its cognate methods are insensitive to the sequential order of the input items, and therefore relies on extra process such as adding positional embeddings to the input sequence to make the model aware of the information contained in the order of sequence. However, existing self-attention methods, combining item sequence and positional embeddings from two heterogeneous data types, may interrupt the underlying semantics of item embeddings [47]. Second, the time complexity of self-attention is quadratic to the length of the input item sequence, which increases unneglectable computational cost compared to RNN/LSTM based SRS for large-scale recommender systems. Third, incorporating self-attention in recommender systems typically leads to huge amounts of model parameters, which may result in a bottleneck in model optimization and an increased chance of over-fitting.

Recent advances in **Multi-layer Perceptron** (**MLP**) architectures, such as MLP-Mixer, gMLP, resMLP, and SMLP [18, 29, 31, 32], show competitive performances in computer vision tasks despite their architectural simplicity and linear computational complexity. This questions the necessity of self-attention mechanisms and shows the possibility of replacing them with simple MLP architectures. To address the aforementioned challenges of self-attention based SRS, our previous study [15] proposes a simple yet effective **MLP** framework for sequential **Rec**ommendations (**MLP4Rec**) as shown in Figure 1, which has two-fold advantages. First, along with the aforementioned MLP-based models, MLP4Rec is by design sensitive to the order of input item sequence, avoiding the bottleneck caused by using positional embeddings. Second, upon pure MLP blocks, MLP4Rec possesses linear computational complexity and a significantly lower amount of model parameters than those self-attention based SRS models.

Specifically, due to the design of bi-directional mixer in existing MLP architectures [31], utilizing them for sequential recommendations can only capture the sequential and cross-channel

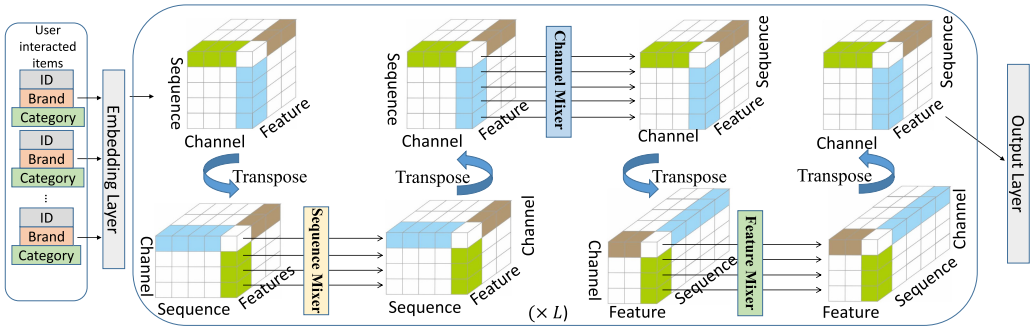

Fig. 1. Overall framework of MLP4Rec.

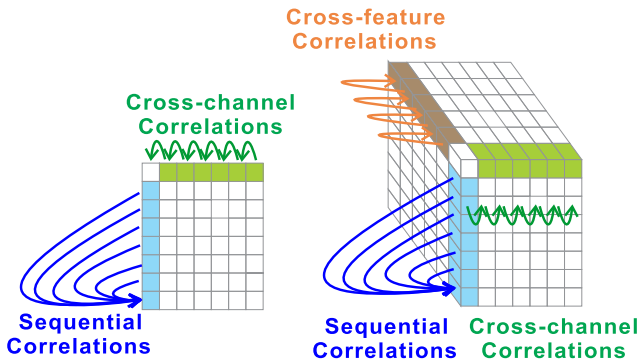

Fig. 2. Bi-directional correlations v.s. Tri-directional correlations .

dependencies, but ignore the cross-feature dependencies, which represent the abstract properties of each channel in an item embedding. To this end, we devised a novel tri-directional information fusion scheme for MLP4Rec with a cross-feature mixer, which enables the framework to capture the complicated interactions among all features of different item embedding channels, as illustrated in Figure 2. In addition, the tri-directional scheme also applies the classic bi-directional mixers from MLP-based models [11, 31] on item explicit features, which learns the users' sequential preferences within these features. Extensive experiments demonstrate that MLP4Rec achieves significantly superior performance than the SOTA methods on two benchmark datasets.

Nevertheless, MLP4Rec still suffers from multiple limitations. Specifically, there are three particular operations that could significantly reduce the efficiency and effectiveness of information extraction, i.e., (i) the cascading mixer structure of MLP4Rec, (ii) the numerous normalization layers between different mixer layers, and (iii) the noise generated by the improper combination order of these operations. In turn, these drawbacks result in inefficient model training and restrict the model's predictive ability. To solve these issues, in this article, a novel **S**parse **MLP** framework for sequential **Rec**ommendations, namely **SMLP4Rec**, is proposed. The new framework improves MLP4Rec's flawed cascading structure with a parallel mode, and removes the normalization layers among mixer layers but places them in front of the framework's main architecture, so as to minimize the impact of their limitations on the model efficiency, while maximizing their utility. As a result, SMLP4Rec's training efficiency and prediction performance have vastly improved in comparison to MLP4Rec. Through extensive experiments, we demonstrate that SMLP4Rec

performs significantly better than SOTA methods, including our previously proposed MLP4Rec on two benchmark datasets. To summarize, this article has the following contributions:

- We investigate the possibility of replacing the self-attention mechanism with simple MLP architectures for sequential recommendations.
- To the best of our knowledge, the MLP4Rec model we proposed is the first work that proposes a tri-directional mixing MLP architecture. In this article, for training acceleration and performance enhancement, a superior model SMLP4Rec is proposed.
- The performance and efficiency improvements of our proposed framework SMLP4Rec are validated via extensive experiments on three benchmark datasets.

2 FRAMEWORK

In this section, we discuss the framework, methodology, and optimization of our previously proposed MLP4Rec framework and our extended framework SMLP4Rec. We will first introduce the MLP4Rec framework, then introduce our new framework SMLP4Rec and its training, inference steps, and model analysis. Finally, a discussion between MLP4Rec, SMLP4Rec, and other related models is provided to better illustrate the model improvement details and the differences among these models.

2.1 Notation Definition

Following commonly adapted settings [6, 17], we denote the participant of interactions—users as $U = \{u_1, \ldots, u_n \ldots, u_N\}$, where n indicates the **item identifier** (**ID**) of the user. Items as $I = \{i_1, \ldots, i_m \ldots, i_M\}$, where m indicates the ID of the item. In addition, each item has some associated features, such as category and brand, we denote those features as $Q = \{q_1^m, \ldots, q_f^m, \ldots, q_F^m\}$, where q_f^m refers to the fth feature of item m. We sort the items that users have interacted with into sequences, thus each user has a corresponding sequence containing items (s)he once viewed chronologically. We denote the item sequence of user n as $S_n = \{i_1, \ldots, i_s \ldots, i_S\}$, where i stands for item, s describes the chronological order of item, and S is the maximum length of the sequence.

2.2 MLP4Rec Framework Overview

Following Figure 1, this section simply summarizes the overall framework of our previously proposed MLP-based SRS model MLP4Rec that is capable of explicitly learning tri-directional information, i.e., sequence, feature, and channel information from the interaction sequences.

As we mentioned before, to make a reasonable prediction, a model must be able to capture the following three-fold information. The first fold is the temporal information, i.e., sequential dependencies among S_n. The second fold refers to the shared information across different explicit characteristics of a feature in the item, i.e., the cross-channel correlation in an interaction matrix. Since different channels (dimensions) of a feature embedding represent different semantics, the cross-channel correlation is also important for the recommendation task. The third fold is the correlations among features of an item, which collectively contribute to modeling the semantic meaning of an item. By repetitively transposing and applying MLP blocks in different directions of the input embedding tensor as shown in Figure 1, we show that MLP4Rec can capture the sequential, cross-channel, and cross-feature correlations step by step. To be specific, MLP4Rec consists of L layers, where each layer has an identical setting, a sequence-mixer, a channel-mixer, and a feature-mixer. Each mixer block contains a normalization operation and two MLP operations with one activation function in the middle for information extraction on a specific dimension of the input matrix. Following [31], all L layers share the same parameters to reduce model parameters. Within each layer, independent sequence-mixers and channel-mixers for different are first applied to learn

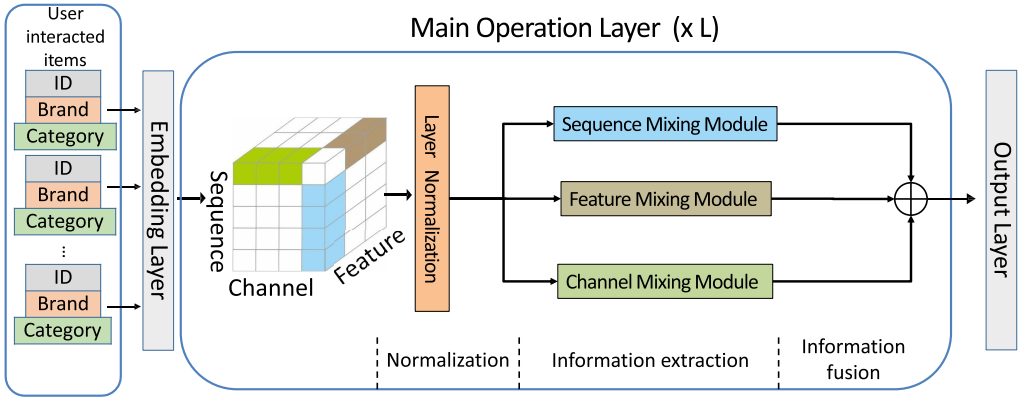

Fig. 3. Framework overview of SMLP4Rec. L is a hyper-parameter controlling the number of main operation layers in the model, and the parameters among these L layers are shared.

the unique characteristics in the sequence and channel-related dimensions. Then, a feature-mixer is utilized to learn correlations among all features in the feature-related dimension. By doing so, MLP4Rec could aggregate the sequential, cross-channel, and cross-feature correlations in a cascading structure in one layer, and MLP4Rec could thus extract useful information for recommendation by applying L such layers.

2.3 Extension: SMLP4Rec Framework

In this subsection, we propose an extended version of the MLP4Rec framework, SMLP4Rec, to optimize the training efficiency, and further obtain performance improvement. In the following paragraphs, we first focus on the limitations of the MLP4Rec, and then introduce the detailed structure of our new framework SMLP4Rec.

2.3.1 Limitations of MLP4Rec. A major problem of MLP4Rec is that in one mixer layer of MLP4Rec, as illustrated in Figure 1, the cascading structure of sequence-mixer, channel-mixer, and feature-mixer will result in the sequential data modification occurring along with each of the three above mixers. In other words, the next mixer will be affected by the information mixed in the previous mixer, making them cannot aggregate the important information of their own dimension independently. This may lead to the inefficient use of intra-dimension information (i.e., the common characteristics in one dimension), and inter-dimension information (i.e., the common characteristics among different dimensions), resulting in inefficient information extraction and fusion.

Another problem comes from the low optimization efficiency of MLP4Rec due to the cascading structure among mixers mentioned above, and the numerous normalization operations in each mixer, which not only reduces the model's training efficiency but also degrades its performance. To solve the above problems, in this article, an extended framework SMLP4Rec is designed.

2.3.2 SMLP4Rec Framework Overview. As shown in Figure 3, the overall framework of our extended work SMLP4Rec could be divided into three parts: (*i*) an embedding layer, (*ii*) multiple main operation layers, with each layer containing normalization operation, information extraction operation, and information fusion operation, and (*iii*) an output layer. The embedding layer and output layer are exactly the same as their counterparts in MLP4Rec. Similar to MLP4Rec, SMLP4Rec also consists of L main operation layers with several mixing modules, and the operations in them are described in detail below.

2.4 Detailed Architecture of SMLP4Rec

2.4.1 Embedding Layer. We adopt a commonly used method for constructing item ID embeddings and feature embeddings, i.e., learning an embedding lookup table to project the discrete item identifiers (i.e., IDs) and explicit features (e.g., category and brand) into dense vector representations with dimension C [1] i.e., the length of the channel dimension. After the embedding layer, we can stack the embeddings of item IDs and explicit features into individual embedding tables, where the row of the embedding table is each embedding vector with a total of F features, and the column of the embedding table contains channel information with length C. Stacking all embedding tables of S items in a sequence together, we obtain a 3-d embedding table as shown in Figure 3.

2.4.2 Normalization Operation. In MLP4Rec's main operation layer, the normalization operations on the sequence and feature mixers may lead to slower training efficiency and excessive inter-dimension interference caused by padding information. Therefore, in SMLP4Rec, the normalization operation is only applied to the channel embedding dimension before the mixing modules in the information extraction operation. This also provides a better data distribution to represent hidden information from different features in different items, as frequent normalization on different dimensions may cumulatively increase the noise in other dimensions. Specifically, the normalization operation can be expressed as follows:

$$\boldsymbol{x_n} = LayerNorm(\boldsymbol{x}), \tag{1}$$

where $\boldsymbol{x}$ is the input of the model, and $\boldsymbol{x_n}$ is the output of the normalization operation and also the input of the information extraction operation.

2.4.3 Information Extraction Operation. Inspired by SMLP's [29] sparse MLP structure and MLP4Rec's mixer structure for information extraction, this information extraction operation block aims to capture the internal relationship among sequence, feature, and channel characteristics in the input matrix simultaneously. Specifically, sequence, feature, and channel information correspond to different dimensions of the input matrix, and this structure aims to help the model understand the data correlation at different positions within the corresponding dimension through its three branches.

Sequence-Mixing Branch: The sequence-mixing branch is an MLP block, which aims to learn the sequential dependencies across the entire item sequence. The sequence-mixing branch takes the rows of the sequence dimension as inputs (applied to the transposed embedding table) and outputs vectors with the same dimension. In this output table, all the sequential dependencies are fused within each other. More specifically, we denote S as the length of the item sequence, F as the number of the item features, and C as the number of channels of each feature (i.e., the embedding size). As shown in Figure 4, the input of the sequence-mixing branch would then be represented as $\boldsymbol{x}_S = \{\boldsymbol{x}^o | o \in C \times F\}$ and $\boldsymbol{x}^o = [\boldsymbol{x}_1^o, \ldots, \boldsymbol{x}_s^o, \ldots, \boldsymbol{x}_S^o]$. o represents dimensions other than the sequence dimension so here $o \in C \times F$. The correlation among $[\boldsymbol{x}_1^o, \ldots, \boldsymbol{x}_s^o, \ldots, \boldsymbol{x}_S^o]$ are sequential, which shows the evolvement of user interest across time, thus making the sequence-mixing branch sensitive to the sequential order. Formally, we denote the output of the sequence-mixing branch at layer l as $\boldsymbol{y}_S$ with its elements:

$$\boldsymbol{y}^o = \boldsymbol{W}^2 g^l(\boldsymbol{W}^1 \boldsymbol{x}^o) \tag{2}$$

g^l is the non-linear activation function at layer l, $\boldsymbol{W}^1 \in \mathbb{R}^{r_S \times S}$ denotes the learnable weights representing the first fully connected layer in the sequence-mixing branch, $\boldsymbol{W}^2 \in \mathbb{R}^{S \times r_S}$ signifies the learnable weights of the second fully connected layer in the sequence-mixing branch, r_S is a

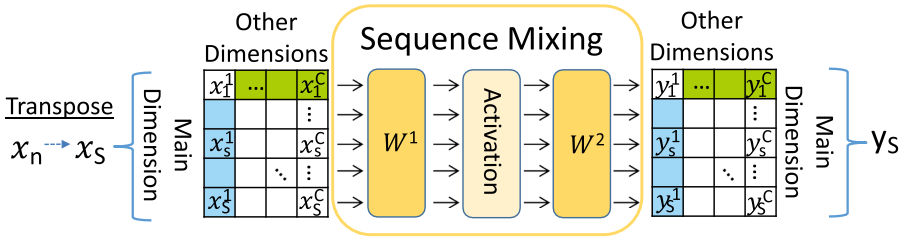

Fig. 4. Architecture of the sequence-mixing branch. For ease of understanding, the feature dimension F of the matrix is hidden.

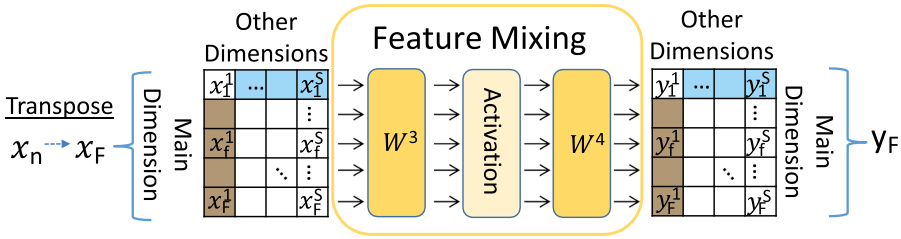

Fig. 5. Architecture of the feature-mixing module. For ease of understanding, the channel dimension C of the matrix is hidden.

tunable hidden size which could be expressed as follows:

$$r_S = \alpha \cdot S, \tag{3}$$

where α is an integer hyper-parameter named expansion factor.

Feature-Mixing Branch: Like the sequence-mixing branch, the feature-mixing branch is also an MLP block with a similar macro architecture. Their key distinction is between their purpose. The objective of the feature-mixing branch is to learn the correlation between different item features. As shown in Figure 5, the feature-mixing branch can thus fuse cross-feature correlation into the representation of each sequence by applying MLP on the feature dimension F. We denote the transposed input $\boldsymbol{x}_F = \{\boldsymbol{x}^o | o \in S \times C\}$ and $\boldsymbol{x}^o = [\boldsymbol{x}_1^o, \ldots, \boldsymbol{x}_f^o, \ldots, \boldsymbol{x}_F^o]$. o represents dimensions other than the feature dimension so here $o \in S \times C$. The output of the feature-mixing branch at layer l is represented as $\boldsymbol{y}_F$ with its elements:

$$\boldsymbol{y}^o = \boldsymbol{W}^4 g^l(\boldsymbol{W}^3 \boldsymbol{x}^o), \tag{4}$$

where $\boldsymbol{W}^3 \in \mathbb{R}^{r_F \times F}$ denotes the learnable weights of the first fully connected layer in the feature-mixing branch, $\boldsymbol{W}^4 \in \mathbb{R}^{F \times r_F}$ is the learnable weights of the second fully connected layer in the feature-mixing branch, and r_F is the tunable hidden size in the feature-mixing branch:

$$r_F = \alpha \cdot F \tag{5}$$

Channel-Mixing Branch: The embedding of an item ID or item feature usually expresses some latent semantics on each dimension. Learning their representation and internal correlation is crucial for recommendations. So the objective of the channel-mixing branch is to learn the correlation within the embedding vector of a feature using an MLP structure. The channel-mixing branch takes the columns of the embedding table as input, as shown in Figure 6. We denote the transposed input $\boldsymbol{x}_C = \{\boldsymbol{x}^o | o \in F \times S\}$ and $\boldsymbol{x}^o = [\boldsymbol{x}_1^o, \ldots, \boldsymbol{x}_c^o, \ldots, \boldsymbol{x}_C^o]$. o represents dimensions

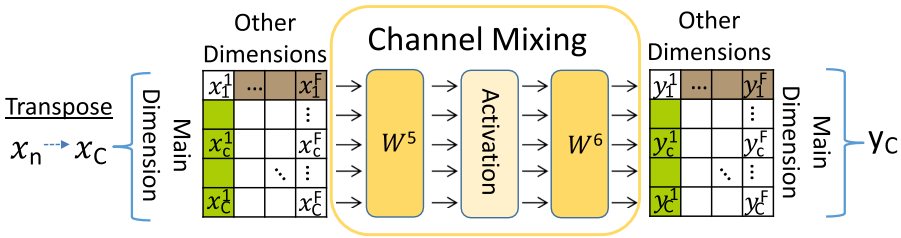

Fig. 6. Architecture of the channel-mixing module. For ease of understanding, the sequence dimension S of the matrix is hidden.

other than the channel dimension so here $o \in F \times S$. The output of the channel-mixing branch at layer l could be represented as $\boldsymbol{y}_C$ with its elements:

$$\boldsymbol{y}^o = \boldsymbol{W}^6 g^l(\boldsymbol{W}^5 \boldsymbol{x}^o), \tag{6}$$

where $\boldsymbol{W}^5 \in \mathbb{R}^{r_C \times C}$ denotes the learnable weights of the first fully connected layer in the channel-mixing branch, $\boldsymbol{W}^6 \in \mathbb{R}^C \times r_C$ is the learnable weights of the second fully connected layer in the channel-mixing branch, and r_C is the tunable hidden size in the channel-mixing branch:

$$r_C = \alpha \cdot C \tag{7}$$

2.4.4 Information Fusion Operation. Finally, the information fusion operation utilizes simple addition to aggregate the information from the three aspects as the output of this layer. The information fusion operation could be expressed as follows:

$$\boldsymbol{y}_{fusion} = Trans(\boldsymbol{y}_S) + Trans(\boldsymbol{y}_F) + Trans(\boldsymbol{y}_C), \tag{8}$$

where $Trans()$ is the transpose operation. By doing so, the model is able to finally capture the cross-dimension information for the recommendation.

2.5 Training and Inference

2.5.1 Training. We adapt Cross-Entropy loss as the loss function for SMLP4Rec:

$$L = -\sum_{S_n \in Seq} \sum_{s \in [1,\ldots,S]} [log(\sigma(r_{i_s,s})) + \sum_{j \notin S_n} log(1 - \sigma(r_{i_j,s}))], \tag{9}$$

where σ demotes sigmoid function, $r_{i_s,s}$ is model's predicted similarity to ground-truth item i_s, and $r_{i_j,s}$ is the predicted similarity to sampled items at position s, j is the negative sampled items, Seq is the set of all users' interaction sequences.

Next, we introduce the optimization algorithm of SMLP4Rec through pseudo-code in Algorithm 1. The PyTorch implementation could be found in our code submission. As shown in Algorithm 1, the structure of SMLP4Rec is extremely simple and similar to MLP4Rec, and the main blocks only perform simple matrix multiplications, therefore maintaining linear complexity.

2.5.2 Inference. We adopt the most commonly used inference method in SRS for fair comparison [6, 41]. To be specific, after L layers of sequence-mixer, channel-mixer and feature-mixer in MLP4Rec or L layers of SMLP blocks in SMLP4Rec, we obtain a sequence of hidden states that contains the sequential, cross-channel, and cross-feature dependencies of each interaction, respectively. Assuming at time step t, we wish to predict next item i_{t+1}, given sequence of hidden states $H = h_1, \ldots, h_S$ with h_S related to i_{t+1}, we can calculate the cosine similarity between h_S and all

ALGORITHM 1: Optimization Algorithm of SMLP4Rec

Input: Users' historical interaction data *Seq*
Output: Well-trained model f

```
Randomly initialize parameters of model f
for Epoch in 1, ..., max epochs do
    for Batch 1, ..., batch number do
        Sample training batch data S_n from Seq
        Generate predictions from f(S_n)
        Calculate loss L based on Equation (9)
        Update model parameters of f via minimizing L
    end for
    if Converged then
        return f
    end if
end for
return f
```

candidates items E_m via dot product as follows:

$$r_{m,S} = h_S \cdot E_m^T \tag{10}$$

where $m = 1, \ldots, M$, $E_m \in \mathbb{R}^{M \times C}$ is the item embedding of all candidate items and $r_{m,S}$ indicates the similarity between hidden state S to all candidate items, top predictions will be ranked by their similarity using this equation.

2.6 Complexity Analysis

2.6.1 Time Complexity Analysis. Since each block of SMLP4Rec is simply an MLP with one hidden layer, their time complexity is simply two matrix multiplications, i.e., $i \times r + r \times k$, where i is the number of input units, r is the number of hidden units, k is the number of output units. Since the hidden units is a constant that is independent of the input or output units, and in SMLP4Rec the number of input units is equal to the output unit, then we can denote the overall complexity as $O(max(S, F, C))$ since we can compute the matrices related to sequence, channel and feature in parallel, which is linear to the maximum sequence length S, feature number F and embedding size C. Compared to the complexity of MLP4Rec which is $O(S+F+C)$ and the self-attention mechanism, which is $O(S^2C + C^2S)$, we can conclude that the theoretical upper bound of the time complexity of the proposed SMLP4Rec is lower.

2.6.2 Space Complexity Analysis. The learnable parameters in SMLP4Rec consist of the parameters from the mixing branches, embedding layer, and output layer. In this part, we will analyze the number of parameters from the mixing branches of the model. For each MLP block, the number of trainable parameters is essentially 2 matrices, their combined size is $i \times r + r \times k$. In addition, for each feature, SMLP4Rec trains separate sequence-mixing branch and channel-mixing branch, so the total space complexity of SMLP4Rec is $O(F(S + C) + F)$, which is the same as MLP4Rec. It's clear that the major influence is F, which is the number of features. However in reality F usually tends to be relatively small, especially when it could be reduced by feature selection. On the other hand, the space complexity of self-attention is $O(SC+C^2)$ [6], which is quadratic to the embedding size. In the experiment part, we show that not only do we keep a theoretically lower upper bound in space complexity, but in practice, we also achieve a significantly lower number of parameters than the baseline models in this article.

2.7 Discussion

In this subsection, we conclude the comparison between MLP-Mixer, MLP4Rec, SMLP4Rec, and SMLP to further illustrate the incremental improvement of MLP4Rec and SMLP4Rec and explain why SMLP4Rec should have the above structure.

2.7.1 Comparison between MLP-Mixer and MLP4Rec. Following our previously studies [15], MLP4Rec is inspired by MLP-Mixer, so distinguishing the difference and connection between MLP4Rec and MLP-Mixer helps to better understand structures in MLP4Rec. When items that users interacted with have no features but their IDs, MLP4Rec is equivalent to the MLP-Mixer. However, this is not a common scenario since, in most of the recommendation settings, items have multiple features. Nevertheless, a vanilla MLP-Mixer can also leverage item features for recommendation, MLP-Mixer was designed for image classification, and we can view the RGB channel of images as item features. MLP-Mixer first divides a 3-dimensional image ($height \times width \times channel$) into several non-overlapping patches, and then projects an individual 3-dimensional patch into a one-dimensional embedding vector and stacks the embedding vectors of different patches. This embedding table is then formulated as input.

Such an approach might work when cross-feature (channel) information is highly homogeneous such as different RGB channels for the same pixel. However, the cross-feature information for an item could be extremely heterogeneous. For instance, the embedding of the textual description of an item and the embedding of the brand of an item, that information has extremely diverged semantics. Moreover, in reality, an item could have a large number of features, all of which can be heterogeneous from each other, so simply projecting all of them into an embedding while preserving the underlying semantics or pattern of each feature could be challenging. Therefore, the key distinction between MLP4Rec and MLP-Mixer is the proposed feature-mixer layer.

2.7.2 Comparison between MLP4Rec and SMLP4Rec. One major change in SMLP4Rec is the main structure of sequence, channel, and feature-oriented information extraction branches, i.e., the Sequence-Mixing, Channel-Mixing, and Feature-Mixing operations. In SMLP4Rec, in order to better aggregate the information of each dimension and avoid the noise introduced into other dimensions due to the sequence of operations between dimensions, the three mixing branches are designed to be parallel rather than serial.

Figure 7 shows the comparison of receptive fields of main layers in SMLP4Rec and MLP4Rec. As shown in the figure, the receptive field of one layer in MLP4Rec is the entire information matrix, while the receptive field of one layer in SMLP4Rec is the stereoscopic cross receptive field formed by overlapping three bar receptive fields. In the following experiments, we demonstrate that SMLP4Rec's receptive field is more advantageous for information extraction. On one hand, for a single layer, the three-dimensional cross receptive field limits the range of information extraction to the highly correlated information to extract more useful common features. For MLP4Rec, although the global receptive field can extract more information in a single layer, its largely different initial weights on different positions and a too large receptive field may lead to a low information extraction efficiency and effectiveness. On the other hand, when the number of layers reaches 3 or above, SMLP4Rec's receptive field could cover the whole matrix [29], that is to say, SMLP4Rec's receptive field would not cause information loss. Instead, compared with MLP4Rec, SMLP4Rec could pay more attention to extracting information with strong relevance.

Moreover, we eliminate the Layer Normalization step from the mixer structures in MLP4Rec and place it at the starting point in each main operation layer. In addition, due to the limited layer numbers in the overall framework, residue connections in the mixer module may cause inefficient parameter updates. Therefore, we also eliminate the residue modules from all mixer

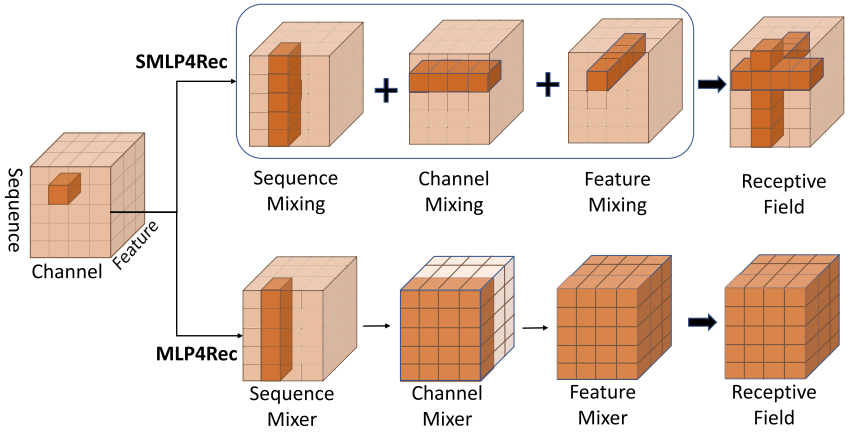

Fig. 7. Receptive field difference of one layer in SMLP4Rec and MLP4Rec.

modules. Finally, the modified three parallel modules (Sequence-Mixing, Channel-Mixing, and Feature-Mixing branches) could represent the information extraction of sequence, channel, and feature parts, respectively.

2.7.3 Discussion: Relation between SMLP and SMLP4Rec. Our information extraction operation contains three branch structures, i.e., the sequence mixing, feature mixing, and channel mixing branches, which are partly inspired by SMLP [29]. As shown in Figure 8(a), SMLP was originally designed to acquire and extract image information with strips of receptive fields in two dimensions, which formed a cross-shaped receptive field on the image. In the sequential recommendation system, however, we apply it to the 3D sequential recommendation data to efficiently extract information using strip receptive fields in three dimensions, as shown in Figure 8(b). The difference is that we apply SMLP on all three dimensions of the information matrix instead of the original two dimensions. This is due to the fact that the information matrix in sequential recommendation is not composed of multiple two-dimensional feature maps like the matrix in computer vision, but has three dimensions of information, i.e., sequence, feature, and channel. Different from SMLP, convolution block is not used in SMLP4Rec, because the 3D matrix in sequential recommendations is not composed of multiple two-dimensional feature maps.

3 EXPERIMENTS

In this section, we will evaluate the performance of SMLP4Rec against different SOTA baselines on three commonly used public benchmark datasets. We also provide our code, evaluation details, and details of the hyper-parameter adjustment.

3.1 Datasets

We choose three widely used datasets to benchmark our performance on both small and large datasets to show the performance of our proposed method on datasets that have relatively fewer features and a dataset that relatively has more features, and their statistics can be found in Table 1.

— **MovieLens**[1]: MovieLens is a site for recommending movies to users given their historical ratings, which is now one of the most commonly used benchmarks across the field of recommender system. We use a smaller dataset ML-100K, and ML-1M which is a bigger one.

[1]https://grouplens.org/datasets/movielens

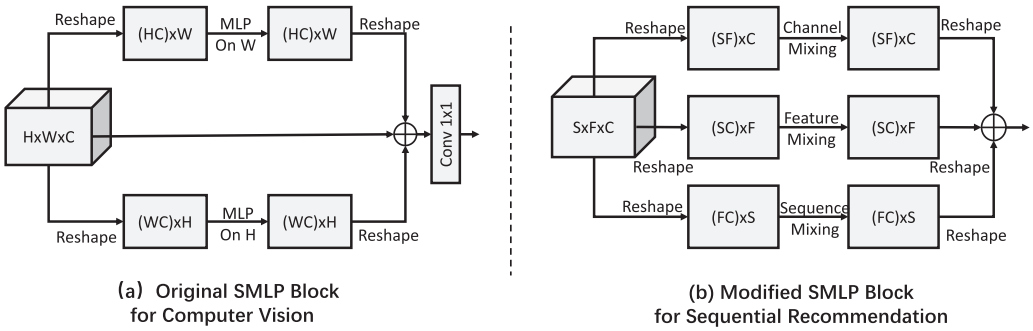

Fig. 8. Structure comparison of original Sparse MLP (SMLP) Block and modified SMLP Block in SMLP4Rec. The original one contains three branches, two for mixing branches of two feature map dimensions, and one for residue connection. The modified one contains three mixing branches for sequence, feature, and channel mixing, with no residue connection or convolutional layers. H for height, W for width, C for channel, S for sequence dimension, and F for feature dimension.

Table 1. Statistics of the Datasets

Data	ML-100K	ML-1M	Beauty
# interactions	100,000	1,000,209	2,023,070
# users	943	6040	1,210,271
# items	1,682	3952	249,274

— **Amazon Beauty**[2]: The online reviews and ratings of Amazon. We use the "Beauty" category in our experiments. We filter out the items and users that have less than 5 interactions for two datasets. We set the maximum sequence length as 50 for both datasets, and conduct zero-padding for shorter sequences.

As for features, we cast away the features that seem less relevant to the prediction empirically, such as "sales type" for Beauty (which only has a single value "Beauty" for all items). In Table 2, we show the names of the features we used, the dataset they belonged to, and their respective types. Textual token sequence refers to a sequence of keywords that indicates items' categories such as ["Action", "Comedy", "Drama"]. We handle these types of features by first using Word2vec [21] to project each of these keywords into individual dense vector representations, and then use average pooling to obtain an embedding that contains the overall semantic. Float token usually refers to features that are expressed in numbers. We use an embedding layer that can be viewed as a linear transformation to project the float token into embedding. And a textual token refers to an individual or group of words, but different from the textual token sequence, the word in a textual token does not have an individual meaning, such as "Louis Vuitton", where "Louis" and "Vuitton" does not represent anything unless together they form a unique token, and we project such token into embeddings.

3.2 Baseline Methods

3.2.1 Model List. In this part, we will introduce the baseline methods from the experiment section of the article in a more detailed manner.

— **PopRec** simply recommends the most popular items to the user and does not account for user interest or item relevance.

[2]http://jmcauley.ucsd.edu/data/amazon/

Table 2. Feature Details

Feature name	Dataset	Feature type
genre	MovieLens	Textual token sequence
movie title	MovieLens	Textual token sequence
release year	MovieLens	Float token
categories	Beauty	Textual token sequence
sales rank	Beauty	Float token
price	Beauty	Float token
brand	Beauty	Textual token

- **BPR** [24] builds matrix factorization model from pair-wise loss function to learn from implicit feedback, it is a classical general recommender system.
- **FPMC** [25] combines Markov Chains and Matrix Factorization method to learn the sequential dependencies in user interaction history as well as users' general preferences.
- **GRU4Rec** [4] uses GRU to improve the performance of traditional RNN in a sequential recommender system.
- **GRU4Rec**$^+$ [5] is an improved version of GRU4Rec, it leverages item features and parallel computation to improve the performance.
- **SASRec** [6] introduces self-attention block for sequential recommendation.
- **SASRec**$^+$ is a naive improved version of SASRec, we append feature embedding of items together with token embedding for a fair comparison with our proposed method, this implementation was reported to have competitive performance in several studies [41, 48].
- **BERT4Rec** [28] uses bidirectional self-attention and mask-language-modelling (Cloze objective) method to make sequential recommendation.
- **FDSA** [41] applies a separate self-attention block on token embedding and feature embedding to learn their correlations.
- **MLP-Mixer**$^+$ [31] is our improved version of MLP-Mixer to make it adapt to sequential recommendation tasks with item explicit features. Specifically, it takes the concatenation of embeddings of item ID and features as input, enabling fair comparison with MLP4Rec. We maintained the overall structure of MLP-Mixer$^+$ and make sure the only structural difference between MLP4Rec and MLP-Mixer$^+$ is the feature-mixer layer.
- **FMLP** [49] is a new baseline model added in our extension work, which uses filters to filter noise in the frequency domain and combines MLP to make recommendations.
- **MLP4Rec** is our previously proposed method for SRS. It uses a serial structure to process information extraction with sequence, feature, and channel-mixer blocks.

3.3 Evaluation Settings

We employ the commonly used evaluation method in SRS, namely next-item prediction. For dataset splitting, the next-item prediction task uses the last item in an interaction sequence as the test set, the item before as the validation set, and the rest of the items will be used as the training set. Following common settings, we pair 100 negative samples with ground-truth items during prediction [6].

Metrics. We apply three commonly used evaluation metrics in the recommendations, namely **Hit Ratio** (**HR**), **Normalized Discounted Cumulative Gain** (**NDCG**), and **Mean Reciprocal Rank** (**MRR**). All results are averaged on three random seeds.

3.4 Implementation Details

The implementation of MLP4Rec, SMLP4Rec, and all baselines are based on the RecBole library [43], an open-source recommender system library, which allows us to test and compare all methods in a fair environment, and allows our results to be reproduced easily. We tune the hyper-parameters based on the original papers' recommendations. If the original papers did not provide detailed hyper-parameters, we perform hyper-parameter tuning via cross-validation with Adam optimizer [9] and early stop strategy. The detailed hyper-parameter tuning results for different models could be found in Section 3.5. The implementation code of MLP4Rec[3] and SMLP4Rec[4] is available online. Note that for other parameters not mentioned in Section 3.5 and the original papers, we follow the common settings in the RecBole library.

3.4.1 Details About Baselines. We note that one of the major concerns is "why are the baseline performances of this study differ from the previous study?" Though we adapt suggested parameters from the models' original paper, we cannot completely replicate their experiment's settings in the RecBole environment, since previous works don't have the consistency in their respective experiments, also, not all works release their full settings. In this article, for baselines, MLP4Rec and SMLP4Rec, the training batch size is fixed at 256, the evaluation batch size is fixed at 512, and we then process a hyper-parameter tuning on every baseline with the detail in Section 3.5.

3.4.2 Details About MLP4Rec. Specifically, for MLP4Rec, we use Adam optimizer for optimization, learning rate is set to be 0.0001, $\beta_1 = 0.9$, $\beta_2 = 0.999$ throughout the experiment. The training batch size is fixed at 256. And the expansion factor for Mixer blocks is fixed at 2. Also, since MLP cannot handle different item sequence lengths, we use padding techniques to fill users whose interaction number is less than the maximum sequence length, and use the most recent interactions from users who have more interactions than the maximum sequence length [6]. For a fair comparison, we set the non-linear activation across all models as GELU [3]. Also, for BERT4Rec, mask ratio is an important influence, according to the suggestion of the original paper, we set that to be 0.2 for the ML-100K and ML-1M datasets and 0.6 for the Beauty dataset.

3.4.3 Extensive Details About SMLP4Rec. For details about SMLP4Rec, most of our settings still adopt the above method. However, several changes have been made to obtain better model performance and training efficiency.

- **Bias Removal.** One difference is that we remove the bias related to MLP in the main operation layer. This is because we find that during training, these biases will occupy a high weight in the output of the corresponding block, making the prediction result too little influenced by other parameters. This phenomenon is not conducive to the prediction of differentiated items, and also significantly slows down the learning speed of the model.
- **Hyper-parameter adjustment.** The other difference is the adjustment of the learning rate and expansion factor. Based on the search results, the learning rate is set to 0.0009 for the Beauty dataset and 0.0001 for ML-100K and ML-1M. For the expansion factor, MLP4Rec only sets it to 2, because increasing it will greatly increase the training time. While in SMLP4Rec, the training speed is about 65% faster due to the optimization of the model architecture. This allows us to search for appropriate values within $[1, 6]$. Eventually, the expansion factor is set to 3. Note that other hyper-parameters are the same as in MLP4Rec.

[3]https://github.com/Li-Muyang/MLP4Rec
[4]https://github.com/Applied-Machine-Learning-Lab/SMLP4Rec

Table 3. Hyper-Parameter Search Range

Model types	Hyper parameters	Search range
A, R, M, O	embedding size	[32,64,96,128]
A, R, M	dropout	[0,0.2,0.4,0.5]
A, R, M	weight decay	[0,1e-3,1e-4]
A, R, M	hidden size	[2,4]
A, R	layers	[2,4]
A	heads	[2,4,8]
M	layers	[4,6,8]

3.5 Hyper-Parameter Tuning

Tuning hyper-parameter is arguably one of the most important tasks when we want to evaluate models' performances. To show our effort in tuning baseline models as best as we could while balancing limited computational resources, we list our hyper-parameter searching range and the result of our parameter tuning process. For simplicity, we divide our baseline and proposed methods into the following sub-categories:

— **A** including self-attention-based methods, which are SASRec, SASRec$^+$, BERT4Rec, FDSA.
— **M** including MLP-based models, which are MLP-Mixer, FMLP, our previous study MLP4Rec, and our extended study SMLP4Rec.
— **O** including other methods, which are PopRec, BPR, and FPMC.
— **R** including RNN-based models, which are GRU4Rec and GRU4Rec$^+$.

If the original paper did not provide a guideline for selecting hyper-parameters, we tune them empirically based on their search range shown in Table 3, for validation we encourage you to use the hyper-parameter tuning modules of RecBole. Note that for hidden size, 2 and 4 refer to the ratio of the hidden layer to the input layer. Based on the above settings, we list out the best hyper-parameters combinations from the defined search range in Table 4.

3.6 Performance Comparison

Table 5 summarizes the comparison results, where models above the dashed line consider only item embeddings, while the below models also involve items' side information. From Table 5, we can make the following general observations:

(1) Starting from GRU4Rec, deep learning based methods exceed traditional methods such as BPR by a large margin, suggesting that in sequential recommendations, deep learning models are better at capturing sequential dependencies. This is because the deep learning based methods could better analyze the associations between the items in the sequence through large numbers of, and multiple layers of neurons.
(2) Models that can handle item features (e.g., SASRec$^+$, FDSA) usually outperform those who cannot (e.g., SASRec, BERT4Rec), indicating the item features is an important part of modeling in sequential recommendations.
(3) Improvement over the best baseline is more significant on the larger dataset "Beauty".
(4) Compared to RNN-based models, self-attention models usually have better performances, which can be attributed to self-attention's stronger capabilities in capturing sequential patterns.
(5) MLP-Mixer$^+$ can achieve comparable performance when compared with the SOTA methods such as SASRec and FDSA. This observation serves as a demonstration of how effectively MLP can aggregate information from various dimensions in the sequential recommendation.

Table 4. Best Hyper-Parameter Results

Model	Dataset	embedding size	hidden size	weight decay	dropout	layers	heads
PopRec	MoiveLens	–	–	–	–	–	–
	Beauty	–	–	–	–	–	–
BPR	MoiveLens	64	–	–	–	–	–
	Beauty	128	–	–	–	–	–
FPMC	MoiveLens	96	–	–	–	–	–
	Beauty	128	–	–	–	–	–
GUR4Rec	MoiveLens	128	2	0	0.4	1	–
	Beauty	128	2	0	0.4	1	–
SASRec	MoiveLens	128	4	0	0.2	2	8
	Beauty	64	2	0	0.5	2	4
BERT4Rec	MoiveLens	128	4	0	0.5	4	4
	Beauty	128	4	0	0.5	4	8
FMLP	MoiveLens	64	4	0	0.5	2	2
	Beauty	64	4	0	0.5	2	2
GUR4Rec$^+$	MoiveLens	128	2	0	0.4	1	–
	Beauty	128	2	0	0.4	1	–
SASRec$^+$	MoiveLens	128	4	0	0.5	2	8
	Beauty	128	4	0	0.5	2	8
FDSA	MoiveLens	128	4	0	0.5	2	8
	Beauty	128	4	0	0.5	4	8
MLP-Mixer$^+$	MoiveLens	128	4	0	0.2	4	–
	Beauty	128	4	0	0.5	4	–
MLP4Rec	MoiveLens	128	4	0	0.4	4	–
	Beauty	128	4	0	0.5	4	–
SMLP4Rec	MoiveLens	128	4	0	0.5	4	–
	Beauty	128	4	0	0.5	4	–

(6) SMLP4Rec and MLP4Rec constantly outperform all baselines including FMLP and MLP-Mixer$^+$ with a significant margin, which suggests that tri-directional information extraction and fusion is an important improvement, which jointly captures sequential, cross-channel, cross-feature correlations.
(7) The performance of the SMLP4Rec model is higher than that of MLP4Rec, which indicates that a step-by-step operation of normalization, parallel information extraction, and information fusion are useful for performance improvement. This is because in SMLP4Rec, each mixing branch can only process information related to its corresponding dimension as shown in Figure 7. This characteristic allows SMLP4Rec to focus more on the common aspects of information within the same dimension, while reducing the impact of noise generated by the unrelated information in other dimensions.
(8) Meanwhile, we compared the training speed of MLP4Rec and SMLP4Rec, which is one of our major improvement points. The experiment is conducted on an NVIDIA GeForce RTX 3060 GPU with a maximum epoch of 100 and an early stopping parameter of 10 (epochs), which is commonly used in RecBole. Under this configuration, the model can automatically terminate training in most cases to achieve optimal performance with the current parameters. We run each experiment 10 times with the optimal parameters searched and report the average result in Table 6. As shown in Table 6, the architectural improvement in the SMLP4Rec framework, with the reduction of normalization operations and successful modeling with parallel structure, brings significant improvement in training efficiency for both per-epoch training time and total training time. The efficiency increase in total training

Table 5. Overall Performance Comparison on Two Datasets, where Best Baseline Performances are Underlined

Methods	ML-100K			ML-1M			Beauty		
Metrics	MRR	NDCG	HR	MRR	NDCG	HR	MRR	NDCG	HR
PopRec	0.1496	0.0783	0.4044	0.3904	0.4547	0.6599	0.0305	0.0443	0.0954
BPR	0.1479	0.1905	0.3474	0.3684	0.4425	0.6781	0.1310	0.1725	0.3055
FPMC	0.1220	0.1813	0.3810	0.3413	0.4165	0.6568	0.1431	0.1838	0.3165
GRU4Rec	0.1860	0.2550	0.4758	0.4015	0.4658	0.6709	0.1632	0.2050	0.3417
SASRec	0.1901	0.2612	0.4920	0.4100	0.4720	0.6697	0.2009	0.2447	0.3874
BERT4Rec	0.1819	0.2568	0.5061	0.3938	0.4572	0.6593	0.1313	0.1738	0.3135
FMLP	0.1809	0.2574	0.5070	0.3932	0.457	0.6606	0.1449	0.1891	0.3336
GRU4Rec$^+$	0.1880	0.2550	0.4758	0.4078	0.4715	0.6742	0.1848	0.2294	0.3746
SASRec$^+$	0.2022	0.2710	0.4970	0.4274	0.4901	0.6894	0.2045	0.2488	0.3930
FDSA	0.1913	0.2625	0.4984	0.4107	0.4753	0.6801	0.2056	0.2522	0.4040
MLP-Mixer$^+$	0.1987	0.2671	0.4920	0.411	0.477	0.6871	0.2089	0.2556	0.4065
MLP4Rec	0.2027	0.2747	0.5118	0.4312	0.4938	0.6919	0.2139	0.2654	0.4326
SMLP4Rec	**0.2189***	**0.2882***	**0.5154***	**0.4322***	**0.4955***	**0.6969***	**0.2153***	**0.2685***	**0.4411***

"*" indicates the statistically significant improvements (i.e., two-sided t-test with $p < 0.05$) over the best baseline. Metrics are measured under the scope of 10, i.e., MRR@10, NDCG@10, and HR@10.

Table 6. Training Time and Speed Comparison between MLP4Rec and SMLP4Rec

Model	Training Time (Second / Epoch)			Total Training Time (Minute)		
	ML-100K	ML-1M	Beauty	ML-100K	ML-1M	Beauty
MLP4Rec	36	53	78	30	51	39
SMLP4Rec	22	32	47	19	33	25
Speed Improvement	**63.64%**	**65.63%**	**65.96%**	**57.89%**	**54.54%**	**56.00%**

time for the same model is smaller than for each epoch. This is due to the inclusion of constant time-consuming processes, such as the time used to save results and checkpoints.

3.7 Model Parameter Quantity Analysis

As shown in Table 7, despite MLP4Rec and SMLP4Rec's superior performance, they also surpass baselines in terms of memory efficiency. Fewer parameters make the MLP4Rec and SMLP4Rec easier to train, but also reduce the risk of over-fitting [11]. Compared to MLP4Rec, SMLP4Rec has a similar parameter quantity, but the design of the parallel mixing branches with fewer regularization layers gives SMLP4Rec better performance and efficiency.

3.8 Hyper-Parameters Analysis

In order to find out the influence of hyper-parameters on MLP4Rec and SMLP4Rec, we conduct the hyper-parameters analysis in this section. Figure 9 shows the influence of layer depth and embedding size on SMLP4Rec and MLP4Rec. From the figure, we could conclude that the performance change trend of SMLP4Rec is similar to MLP4Rec. In terms of overall performance, SMLP4Rec is superior to MLP4Rec, especially when the layer depth is equal to 4 and the embedding Size is equal to 128. This may be because when these two parameters are small, two models haven't fully extracted information from the dataset, and when the parameters are large, both two models may cause overfitting. When the parameters of these models are appropriate, a framework with more

Table 7. Model Parameter Quantity Analysis on Beauty Dataset

Model	Param	NDCG@10	HR@10
BERT4Rec	2.3M	0.1738	0.3135
GRU4Rec$^+$	2.5M	0.2294	0.3746
SASRec$^+$	2.0M	0.2488	0.3930
FDSA	3.2M	0.2522	0.4040
MLP-Mixer$^+$	1.8M	0.2556	0.4065
MLP4Rec	**1.7M**	**0.2654**	**0.4326**
SMLP4Rec	**1.7M**	**0.2685**	**0.4411**

"Param" refers to the number of trainable model parameters.

Table 8. Ablation Study Comparison of SMLP4Rec

Model	MRR@10	NDCG@10	HR@10
w/o S-Mixing	0.1767	0.2212	0.3665
w/o F-Mixing	0.1975	0.2467	0.4067
w/o C-Mixing	0.2066	0.2552	0.4132
SMLP4Rec	**0.2153**	**0.2685**	**0.4411**

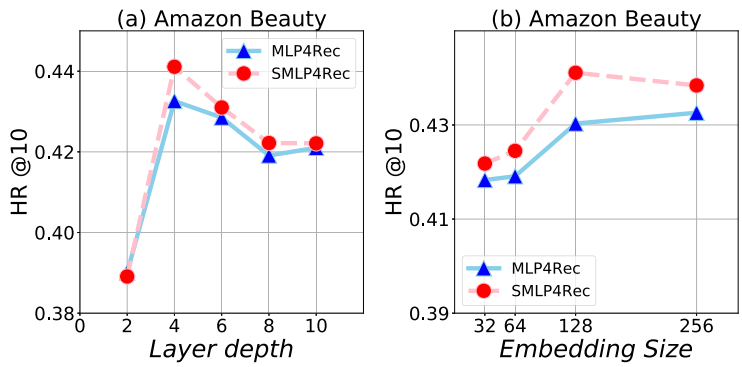

Fig. 9. Influence of hyper-parameters on model performance.

orderly feature extraction and fusion operation and fewer but effective normalization operations could help SMLP4Rec better sort out useful information and alleviate excessive interference from invalid inter-dimensional information or noise.

3.9 Ablation Study

Here, we investigate the necessity of each mixing branch structure, namely the sequence, channel, and feature mixing branch by answering the questions:

— *Q: What is the contribution of each respective module in SMLP4Rec?*

To answer this question, we design the following alternatives to SMLP4Rec:

— **w/o S-Mixing** is the simplified SMLP4Rec without the Sequence Mixing branch.
— **w/o C-Mixing** is the simplified SMLP4Rec without the Channel Mixing branch.
— **w/o F-Mixing** is the simplified SMLP4Rec without the Feature Mixing branch.

The performances of alternatives are displayed in Table 8. The table addresses the above question Q by showing that:

(1) Without the Sequence-Mixing branch, SMLP4Rec's performance degenerates most significantly, indicating that sequential information is vital in the sequential recommendation. Without the sequential information, the recommendation model is unable to capture the user's interaction history, let alone establish the relationship between the items purchased by the user. This leads to the model degenerating into a generic recommendation paradigm, resulting in a significant drop in performance.
(2) Without the Feature-Mixing branch, the performance of SMLP4Rec also suffers a significant decrease in performance, indicating that the model without this branch cannot directly extract inter-feature information from the framework.
(3) Without the Channel-Mixing branch, SMLP4Rec suffers a decrease in performance. The most likely reason is that the model cannot directly update the intra-information for features.
(4) From the perspective of model performance, sequential information is the most critical factor for sequential recommendations, followed by cross-feature information and cross-channel information. However, all three contribute significantly to the performance of the sequential recommendation model. Therefore, it is crucial for SMLP4Rec to accurately and efficiently extract these three aspects of information through its three branches.

4 RELATED WORK

In this section, we review the related work from the literature on sequential recommendation systems, attention structure, and MLP-based Architecture.

4.1 Sequential Recommendation Systems

RNN-based recommendation models can handle complex sequential dependencies in sequential recommendations by compressing previous user–item interactions into a vector to summarize the information, and then predict the next possible interaction [23, 38, 44–46]. For example, GRU4Rec [4] is one of the most representative RNN-based SRS, which implements a **gated recurrent unit** (**GRU**) to improve the modeling of long-term dependencies, however, even with GRU, RNN-based models still cannot perform very well on a long sequence.

In recent years, (self-)attention methods [12, 13, 16, 19, 33] show SOTA performances in SRS. By two-way linking and weighting mechanism. Self-attention structure can capture the sequence information and highlight important parts of it. SASRec [6] is one of the first to implement self-attention for SRS and obtains promising results, by stacking several self-attention blocks, SASRec tends to consider long-term dependencies on dense datasets while focusing on short-term dependencies in sparse datasets. This proved to be critical for the adaptive processing of datasets with different densities, and helped it better extract complex dependencies between items. Inspired by the success of BERT [8] in text understanding, BERT4Rec [28] implements bi-directional self-attention blocks to learn the representations for users' historical behavior sequences and introduces the Cloze task in the training step to train their bidirectional model without information leakage. This work also shows promising results. On the other hand, FDSA [41] uses self-attention on both item tokens and item features to further capture meaningful implicit feature-level transition patterns, which may help a lot for the capture of the full sequential patterns. To ensure better performance and adaptability, CBiT [2] applies a sliding window to the long user sequences to allow for more granular partitioning of user sequences, and then combines the cloze mask and dropout mask to generate high-quality positive samples for contrast learning of multiple pairs of representations.

In addition, many types of research are also devoted to combining self-attention structure with methods such as contrastive learning [37], multi-interest learning [30], and multi-behavior learning [39] to achieve performance improvement. Nevertheless, the drawbacks of self-attention are just as significant. Its positional embeddings may be detrimental to the retention of semantic infor-

mation. Meanwhile, its computational complexity is quadratic to the length of the input sequence and embedding size. These drawbacks make it difficult for deployment.

4.2 Attention Structure

Attention structure [22] is an important part of the sequential-based model. Specifically, it is a weighted sum operation on each row of the input matrix, where each row represents an embedding vector. This operation can be expressed as follows:

$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{Softmax}(\text{Linear}([\boldsymbol{Q}, \boldsymbol{K}])) \cdot \boldsymbol{V} \tag{11}$$

where $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ are linear transformations of the input matrix. By means of weighted summation, it can assign different weights according to the similarity of rows in the input matrix, so as to determine the contribution of different embeddings in information aggregation. Since common embeddings generally do not contain position information, and the attention mechanism itself cannot recognize the position of the embedding, it is usually necessary to add a position token to each embedding when applying the attention mechanism, so as to further obtain the correlation between position information. Since the attention structure was proposed [22], it has been regarded as a very effective structure for capturing sequence features. This argument was accepted by many people especially after Transformer [33, 36] was put forward and proved to be valid on many sequential-related tasks. On the other hand, the large amount of computation required by attention structures and the potential impact brought by the introduction of position embedding have also attracted the attention of researchers. Some researchers try to study variations of attention structure to accelerate its efficiency [7, 26, 35], or position embedding [10, 27, 34] to enhance the effect of attention. Other researchers argue that the attention mechanism may be effective, but it is not necessary. Instead, they hope to capture sequence information using a simple MLP structure [18, 31], which may facilitate the simplification of computation and also eliminate the need to introduce position embedding.

In the field of sequential recommendation, existing self-attention methods, combining item sequence and positional embeddings from two heterogeneous data types, may interrupt the underlying semantics of item embeddings [47]. Second, self-attention methods' computational complexity is quadratic to the length of the input item sequence, which yields an unneglectable computational cost for large-scale recommender systems. Third, incorporating self-attention in recommender systems typically leads to huge numbers of model parameters, which results in difficulty in model optimization and the increased chance of over-fitting. Currently, there has been some work trying to optimize model efficiency by improving the structure of attention and how it is embedded in recommender systems [16, 19]. MLP-based architectures, on the other hand, may avoid these drawbacks with a simpler structure when it comes to sequential recommendations.

4.3 MLP-Based Architectures

Recent development in MLP architectures reveals high potential for computer vision and sequential tasks [14, 18, 29, 31, 32]. Among them, MLP-Mixer [31] is a symbolic example of recent advances in MLP-based models. MLP-Mixer uses a token-mixer and channel-mixer to separately learn the spatial and channel correlations. With linear computation complexity and simpler architectures, MLP-Mixer is reported to have comparable performance compared with SOTA methods. MOI-Mixer [11] is the first work to investigate the possibility of implementing MLP-Mixer in the sequential recommendation. They propose a Multi-Order-Interaction layer to improve vanilla MLP-Mixer's performance and capture the desired kth order interactions while maintaining the linear computational cost of the MLP layer. Similarly, gMLP [18] uses a cross-token MLP structure named Spatial Gating Unit to capture sequential information. Their experiments on image

classification and masked language modeling tasks suggest that self-attention is not a necessary ingredient for scaling up machine learning models, although it can be a useful addition depending on the task.

The above works indicate that the MLP structure has the potential to replace the attention structure for sequence information extraction in sequential tasks. At the same time, since the parameters of the MLP structure depend on their position, it has no requirements for inserting the position embeddings into the input embeddings. On the other hand, its linear complexity also makes it more efficient in processing sequence features than the attention structure, and may need fewer parameters in the same tasks compared with the attention structure. However, it is important to note that the MLP operation can in fact be viewed as a weighted aggregation of a set of one-dimensional value representations. Attention, on the other hand, can aggregate a set of representations of any dimension. Thus, although in this article MLP is shown to be effective on sequential recommendations with each branch for one-dimension information extraction (sequence, feature, or channel), further research is needed to validate whether MLP has the potential to replace attention in other situations where the representations that need to be aggregated are multidimensional matrices.

On the other hand, several other MLP-based models focusing on noise reduction and receptive field research are also mentioned in this article. FMLP [49] combines learnable filters that can attenuate noise in the frequency domain with MLP to enhance the resistance of the recommender system to noise in the frequency domain. SMLP [29] is an MLP-based framework aiming to replace the self-attention module in computer vision. It significantly reduces the number of model parameters and computational complexity by making MLP separately on two dimensions of the feature map to form a crossed receptive field and obtains an excellent feature extraction effect. This approach of MLP operations for specific dimensions and then obtaining information fusion has great potential in sequential recommendation systems. In this article, our extended model SMLP4Rec is partly referred to the SMLP approach.

5 CONCLUSION

In this article, we first introduce a simple but efficient architecture proposed previously with only MLP blocks for sequential recommendations named MLP4Rec. This architecture leverages a novel way to coherently connects sequential, cross-channel and cross-feature correlations in users' historical interaction data to mine their preference.

In order to further improve the model training efficiency and prediction effect of MLP4Rec, in this article, we further propose SMLP4Rec which replaces the flawed cascading structure of MLP4Rec with a parallel manner, and integrates normalization layers to minimize their impact on the model efficiency while maximizing their effectiveness. These improvements enable SMLP4Rec to focus more on the aggregation and extraction of information in the same dimensions, while reducing the mutual interference between irrelevant information. Meanwhile, the parallel model structure and fewer regularization layers greatly improve the model's operation efficiency and increase its usability. As a result, SMLP4Rec's training efficiency and prediction effectiveness are vastly improved in comparison to MLP4Rec.

From a detailed comparison with baselines, we conclude that our models show superior performances against the SOTA methods with a significant margin on three commonly used benchmark datasets Beauty, ML-100K, and ML-1M, validating that:

- They offer a powerful alternative to those current self-attention based methods;
- MLP on feature information enables the proposed model to cope with heterogeneous features and capture their correlations.
- SMLP4Rec and MLP4Rec's simpler model architecture and much fewer model parameters enhance their scalability in large-scale practical recommender systems.

— Compared with our original framework MLP4Rec, our new framework SMLP4Rec has better information extraction efficiency and effect. As a result, it is proven to have better training speed and final prediction results.

REFERENCES

[1] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu and Hemal Shah. 2016. Wide & deep learning for recommender systems. In *Proceedings of the DLRS*.

[2] Hanwen Du, Hui Shi, Pengpeng Zhao, Deqing Wang, Victor S. Sheng, Yanchi Liu, Guanfeng Liu, and Lei Zhao. 2022. Contrastive learning with bidirectional transformers for sequential recommendation. In *Proceedings of the CIKM*.

[3] Dan Hendrycks and Kevin Gimpel. 2016. Gaussian error linear units (GELUs). arXiv:1606.08415v5. Retrieved from https://arxiv.org/abs/1606.08415v5

[4] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2015. Session-based recommendations with recurrent neural networks. arXiv:1511.06939v4. Retrieved from https://arxiv.org/abs/1511.06939v4

[5] Balázs Hidasi, Massimo Quadrana, Alexandros Karatzoglou, and Domonkos Tikk. 2016. Parallel recurrent neural network architectures for feature-rich session-based recommendations. In *Proceedings of the RecSys*.

[6] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *Proceedings of the ICDM*.

[7] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. 2020. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the ICML*.

[8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the AACL*.

[9] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. In *Proceedings of the ICLR (Poster)*.

[10] Shun Kiyono, Sosuke Kobayashi, Jun Suzuki, and Kentaro Inui. 2021. SHAPE: Shifted absolute position embedding for transformers. In *Proceedings of the EMNLP*.

[11] Hojoon Lee, Dongyoon Hwang, Sunghwan Hong, Changyeon Kim, Seungryong Kim, and Jaegul Choo. 2021. MOI-mixer: Improving MLP-mixer with multi order interactions in sequential recommendation. arXiv:2108.07505v1. Retrieved from https://arxiv.org/abs/2108.07505v1

[12] Chengxi Li, Yejing Wang, Qidong Liu, Xiangyu Zhao, Wanyu Wang, Yiqi Wang, Lixin Zou, Wenqi Fan, and Qing Li. 2023. STRec: Sparse transformer for sequential recommendations. In *Proceedings of the RecSys*.

[13] Jing Li, Pengjie Ren, Zhumin Chen, Zhaochun Ren, Tao Lian, and Jun Ma. 2017. Neural attentive session-based recommendation. In *Proceedings of the CIKM*.

[14] Muyang Li, Zijian Zhang, Xiangyu Zhao, Wanyu Wang, Minghao Zhao, Runze Wu, and Ruocheng Guo. 2023. AutoMLP: Automated MLP for sequential recommendations. In *Proceedings of the WWW*.

[15] Muyang Li, Xiangyu Zhao, Chuan Lyu, Minghao Zhao, Runze Wu, and Ruocheng Guo. 2022. MLP4Rec: A pure MLP architecture for sequential recommendations. In *Proceedings of the IJCAI*.

[16] Yang Li, Tong Chen, Peng-Fei Zhang, and Hongzhi Yin. 2021. Lightweight self-attentive sequential recommendation. In *Proceedings of the CIKM*.

[17] Zhi Li, Hongke Zhao, Qi Liu, Zhenya Huang, Tao Mei, and Enhong Chen. 2018. Learning from history and present: Next-item recommendation via discriminatively exploiting user behaviors. In *Proceedings of the KDD*.

[18] Hanxiao Liu, Zihang Dai, David So, and Quoc V. Le. 2021. Pay attention to MLPs. In *Proceedings of the NeurIPS*.

[19] Langming Liu, Liu Cai, Chi Zhang, Xiangyu Zhao, Jingtong Gao, Wanyu Wang, Yifu Lv, Wenqi Fan, Yiqi Wang, Ming He, Zitao Liu and Qing Li. 2023. LinRec: Linear attention mechanism for long-term sequential recommender systems. In *Proceedings of the SIGIR*.

[20] Qidong Liu, Fan Yan, Xiangyu Zhao, Zhaocheng Du, Huifeng Guo, Ruiming Tang, and Feng Tian. 2023. Diffusion augmentation for sequential recommendation. In *Proceedings of the CIKM*.

[21] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient estimation ofword representations in vector space. arXiv:1301.3781v3. Retrieved from https://arxiv.org/abs/1301.3781v3

[22] Volodymyr Mnih, Nicolas Heess, Alex Graves, and Koray Kavukcuoglu. 2014. Recurrent models of visual attention. In *Proceedings of the NeurIPS*.

[23] Massimo Quadrana, Alexandros Karatzoglou, Balázs Hidasi, and Paolo Cremonesi. 2017. Personalizing session-based recommendations with hierarchical recurrent neural networks. In *Proceedings of the RecSys*.

[24] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2009. BPR: Bayesian personalized ranking from implicit feedback. In *Proceedings of the UAI*.

[25] Steffen Rendle, Christoph Freudenthaler, and Lars Schmidt-Thieme. 2010. Factorizing personalized Markov chains for next-basket recommendation. In *Proceedings of the WWW*.

[26] Zhuoran Shen, Mingyuan Zhang, Haiyu Zhao, Shuai Yi, and Hongsheng Li. 2021. Efficient attention: Attention with linear complexities. In *Proceedings of the WACV*.
[27] Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. 2021. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing* 568 (2021), 127063.
[28] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the CIKM*.
[29] Chuanxin Tang, Yucheng Zhao, Guangting Wang, Chong Luo, Wenxuan Xie, and Wenjun Zeng. 2022. Sparse MLP for image recognition: Is self-attention really necessary?. In *Proceedings of the AAAI*.
[30] Yu Tian, Jianxin Chang, Yanan Niu, Yang Song, and Chenliang Li. 2022. When multi-level meets multi-interest: A multi-grained neural model for sequential recommendation. In *Proceedings of the SIGIR*.
[31] Ilya O. Tolstikhin, Neil Houlsby, Alexander Kolesnikov, Lucas Beyer, Xiaohua Zhai, Thomas Unterthiner, Jessica Yung, Andreas Steiner, Daniel Keysers, Jakob Uszkoreit, Mario Lucic, and Alexey Dosovitskiy. 2021. MLP-mixer: An all-MLP architecture for vision. In *Proceedings of the NeurIPS*.
[32] Hugo Touvron, Piotr Bojanowski, Mathilde Caron, Matthieu Cord, Alaaeldin El-Nouby, Edouard Grave, Gautier Izacard, Armand Joulin, Gabriel Synnaeve, Jakob Verbeek, and Hervé Jégou. 2022. ResMLP: Feedforward networks for image classification with data-efficient training. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
[33] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the NeurIPS*.
[34] Benyou Wang, Lifeng Shang, Christina Lioma, Xin Jiang, Hao Yang, Qun Liu, and Jakob Grue Simonsen. 2020. On position embeddings in BERT. In *Proceedings of the ICLR*.
[35] Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. 2020. Linformer: Self-attention with linear complexity. arXiv:2006.04768v3. Retrieved from https://arxiv.org/abs/2006.04768v3
[36] Xin Xia, Junliang Yu, Qinyong Wang, Chaoqun Yang, Nguyen Quoc Viet Hung, and Hongzhi Yin. 2023. Efficient on-device session-based recommendation. *ACM Transactions on Information Systems* 41, 4 (2023), 1–24.
[37] Yuhao Yang, Chao Huang, Lianghao Xia, Chunzhen Huang, Da Luo, and Kangyi Lin. 2023. Debiased contrastive learning for sequential recommendation. In *Proceedings of the WWW*.
[38] Feng Yu, Qiang Liu, Shu Wu, Liang Wang, and Tieniu Tan. 2016. A dynamic recurrent model for next basket recommendation. In *Proceedings of the SIGIR*.
[39] Enming Yuan, Wei Guo, Zhicheng He, Huifeng Guo, Chengkai Liu, and Ruiming Tang. 2022. Multi-behavior sequential transformer recommender. In *Proceedings of the SIGIR*.
[40] Chi Zhang, Yantong Du, Xiangyu Zhao, Qilong Han, Rui Chen, and Li Li. 2022. Hierarchical item inconsistency signal learning for sequence denoising in sequential recommendation. In *Proceedings of the CIKM*.
[41] Tingting Zhang, Pengpeng Zhao, Yanchi Liu, Victor S. Sheng, Jiajie Xu, Deqing Wang, Guanfeng Liu, and Xiaofang Zhou. 2019. Feature-level deeper self-attention network for sequential recommendation.. In *Proceedings of the IJCAI*.
[42] Kesen Zhao, Xiangyu Zhao, Zijian Zhang, and Muyang Li. 2022. MAE4Rec: Storage-saving transformer for sequential recommendations. In *Proceedings of the CIKM*.
[43] Wayne Xin Zhao, Shanlei Mu, Yupeng Hou, Zihan Lin, Yushuo Chen, Xingyu Pan, Kaiyuan Li, Yujie Lu, Hui Wang, Changxin Tian, Yingqian Min, Zhichao Feng, Xinyan Fan, Xu Chen, Pengfei Wang, Wendi Ji, Yaliang Li, Xiaoling Wang, and Ji-Rong Wen. 2021. RecBole: Towards a unified, comprehensive and efficient framework for recommendation algorithms. In *Proceedings of the CIKM*.
[44] Xiangyu Zhao, Long Xia, Jiliang Tang, and Dawei Yin. 2019. Deep reinforcement learning for search, recommendation, and online advertising: A survey. *ACM SIGWEB Newsletter* 2019 (2019), 1–15.
[45] Xiangyu Zhao, Long Xia, Liang Zhang, Zhuoye Ding, Dawei Yin, and Jiliang Tang. 2018. Deep reinforcement learning for page-wise recommendations. In *Proceedings of the RecSys*.
[46] Xiangyu Zhao, Liang Zhang, Zhuoye Ding, Long Xia, Jiliang Tang, and Dawei Yin. 2018. Recommendations with negative feedback via pairwise deep reinforcement learning. In *Proceedings of the KDD*.
[47] Jianqiao Zheng, Sameera Ramasinghe, and Simon Lucey. 2021. Rethinking positional encoding. arXiv:2107.02561v3. Retrieved from https://arxiv.org/abs/2107.02561v3
[48] Kun Zhou, Hui Wang, Wayne Xin Zhao, Yutao Zhu, Sirui Wang, Fuzheng Zhang, Zhongyuan Wang, and Ji-Rong Wen. 2020. S3-Rec: Self-supervised learning for sequential recommendation with mutual information maximization. In *Proceedings of the CIKM*.
[49] Kun Zhou, Hui Yu, Wayne Xin Zhao, and Ji-Rong Wen. 2022. Filter-enhanced MLP is all you need for sequential recommendation. In *Proceedings of the WWW*.

Received 8 January 2023; revised 22 September 2023; accepted 28 November 2023